

Manual: HueMotion

Controlling RGBW lighting through gestures of the hands

Kailani Alarcon

New Product Development

LUX dynamics

6/8/2026

System Purpose

HueMotion is a local gesture-controlled lighting demonstration for LUX Dynamics. It allows a user to stand in front of a webcam, perform a supported hand gesture, and see the matching RGBW lighting scene on the browser interface. When Casambi triggering is enabled, the same gesture also sends a command to a Casambi lighting network.

The system is meant for demonstration, testing, and interactive lighting control. It is not only a visual web demo. In its current uploaded version, the browser can communicate with a local Node.js server, and the server can send real scene commands to Casambi through a WebSocket bridge.

The project has two related versions:

Version	Control Method	Notes
GitHub version	ESA2 plus AutoHotkey	Uses keyboard shortcuts to trigger ESA2 scenes.
Uploaded current version	Casambi WebSocket bridge	Sends Casambi controlScene messages directly.

The Casambi version is the better version to document as the current system because it includes the newer HueMotion interface, the ws dependency, a .env configuration file, and direct scene IDs for Red, Green, Blue, Yellow, White, Purple, Off, and Gradient

Table of Contents

System Purpose	1
High-Level System Description.....	3
Main Use Case	3
Required Hardware	3
Required Software	4
Project Files	4
Recommended Folder Layout.....	5
Configuartion File.....	5
Installation Notes	6
Browser Interface.....	6
Gesture-to-Light Mapping	7
How Gesture Detection Works	7
Gesture Stabilization.....	8
Cooldown.....	8
Casambi Bridge Operation.....	8
Server Routes.....	9
Operator Instructions	10
Demonstration Procedure.....	10
Test Procedure	11
Troubleshooting.....	11
Page does not load	11
Logo does not appear	12
Camera does not start.....	12
Hand tracking does not work	12
Gestures are inaccurate	12
Casambi button works, but lights do not change	12
Server says missing .env values.....	13
Server says Casambi login failed.....	13
Server says Casambi WebSocket is not open	13
Wrong scene triggers	13
Security and Safety Rules	13
Development Notes.....	14
Legacy ESA2 Version Notes	15
References.....	15

High-Level System Description

HueMotion has three main layers.

The first layer is the browser interface. This is the part the operator sees. It displays the LUX Dynamics logo, live camera feed, detected gestures, selected lighting scene, RGBW output preview, system status, and gesture control cards.

The second layer is the gesture recognition logic. This runs inside the browser. It uses MediaPipe HandLandmarker to read hand landmark positions from the webcam feed. The JavaScript then checks finger positions, hand distances, and hand motion to classify gestures.

The third layer is the lighting bridge. This runs in Node.js. The browser calls a local /control route when a recognized gesture should trigger a real lighting scene. The Node.js server validates the requested key, logs into Casambi, opens a WebSocket wire, and sends a scene control message.

Main Use Case

The normal operating sequence is:

1. The operator starts the local server.
2. The operator opens the HueMotion page in a browser.
3. The operator clicks Start.
4. The browser asks for camera permission.
5. The camera feed appears.
6. The operator performs a supported gesture.
7. HueMotion recognizes the gesture.
8. The page updates the gesture name, scene name, scene color, and RGBW output.
9. The operator clicks Enable Casambi when real lighting control is desired.
10. The next valid gesture sends a Casambi scene command.

Casambi triggering is intentionally off at startup. This makes it possible to test the camera and gesture recognition without changing real lights.

Required Hardware

The system requires a computer with a webcam. A built-in laptop camera is enough for basic testing, but a dedicated USB webcam is better for public demonstrations because it gives more control over angle, distance, and placement.

The computer should be connected to the same environment where the Casambi account and network are available. Since the current version uses Casambi cloud endpoints and a WebSocket bridge, the computer also needs internet access.

For best demo results, use a stable webcam mount, even lighting on the user's hands, and a background that contrasts with skin and clothing. Avoid strong backlighting because it can make hand landmarks less stable

Required Software

The Casambi version requires:

Software	Purpose
Node.js	Runs the local server.
npm	Installs project dependencies.
Modern browser	Runs the webcam interface and MediaPipe hand tracking.
Internet access	Loads MediaPipe files and connects to Casambi.
Casambi account access	Allows scene triggering on the configured network.

Chrome or Edge is recommended because they generally handle webcam access, JavaScript modules, and MediaPipe reliably.

The uploaded current version uses the ws Node package for WebSocket communication. Run npm install in the project folder before starting the server.

Project Files

The current uploaded version contains these core files:

File	Purpose
server.casambi.js	Main Node.js server, browser page, gesture interface, Casambi bridge, and control endpoint.
package.json	Lists the required ws dependency.
package-lock.json	Locks the installed dependency version.
.env	Stores private Casambi configuration values. Do not publish.
logo.jpg	LUX Dynamics logo is shown in the interface.

Recommended Folder Layout

Use this layout for the Casambi version:

```
Light-Gesture/  
  server.casambi.js  
  package.json  
  package-lock.json  
  .env  
  logo.jpg  
  .gitignore
```

Recommended .gitignore:

```
.env  
node_modules/  
.DS_Store
```

The .env file should stay local. The node_modules folder should not be committed because dependencies can be restored with npm install.

Configuration File

The .env file stores all private Casambi connection values and scene IDs. Use this template format:

```
PORT=3000  
  
CASAMBI_BASE_URL=https://door.casambi.com  
CASAMBI_WS_URL=wss://door.casambi.com/v1/bridge/  
  
CASAMBI_API_KEY=your_api_key_here  
CASAMBI_EMAIL=your_email_here  
CASAMBI_PASSWORD=your_password_here  
CASAMBI_NETWORK_ID=your_network_id_here  
CASAMBI_WIRE=1  
  
CASAMBI_SCENE_RED=1  
CASAMBI_SCENE_GREEN=2  
CASAMBI_SCENE_BLUE=3  
CASAMBI_SCENE_YELLOW=4  
CASAMBI_SCENE_WHITE=5  
CASAMBI_SCENE_PURPLE=10  
CASAMBI_SCENE_OFF=7  
CASAMBI_SCENE_GRADIENT=12
```

Each scene value must match the scene ID inside the Casambi network. For example, if the Red scene is scene ID 1, then CASAMBI_SCENE_RED=1. If the scene IDs change in Casambi, the .env file must be updated.

Some values may exist in the uploaded .env file but are not actively used by the current server code. For example, values like CASAMBI_LOGIN_MODE, CASAMBI_TARGET_TYPE, and CASAMBI_UNIT_IDS may be useful for future expansion, but the current server mainly depends on API key, email, password, network ID, wire number, and scene IDs.

Installation Notes

Open the project folder in Command Prompt, PowerShell, or a terminal.

Install dependencies:

```
npm install
```

Confirm that node_modules is created.

Confirm that the folder contains:

```
server.casambi.js  
package.json  
package-lock.json  
.env  
logo.jpg
```

Start the server:

```
node server.casambi.js
```

The terminal should show that HueMotion is running at the local server address.

Open the browser to:

```
http://localhost:3000
```

The HueMotion interface should load.

Browser Interface

The interface has three main areas.

The top bar contains the LUX Dynamics logo, the HueMotion title, the developer name, and a system ready indicator.

The left panel contains the live camera view. It includes the video feed, a hand tracking overlay, and three buttons: Start, Stop, and Enable Casambi.

The center panel contains the current detected gesture, the selected lighting scene, the current system status, and the RGBW output preview.

The right panel contains the gesture controls. Each gesture card shows the gesture name and the lighting response. When a gesture is active, the matching card is highlighted.

Gesture-to-Light Mapping

Gesture	Lighting Scene	RGBW Preview	Control Key	Casambi Scene Variable
Heart	Red	R 100%	r	CASAMBI_SCENE_RED
Crying	Blue	B 100%	b	CASAMBI_SCENE_BLUE
Jazz Hands	Yellow	R 100%, G 100%	y	CASAMBI_SCENE_YELLOW
Open Palm	White	W 100%	w	CASAMBI_SCENE_WHITE
Thumbs Up	Green	G 100%	g	CASAMBI_SCENE_GREEN
Peace Sign	Purple	R 100%, B 100%	p	CASAMBI_SCENE_PURPLE
Shaka	Gradient	Cycle preview	x	CASAMBI_SCENE_GRADIENT
Hold Fist	Off	All 0%	o	CASAMBI_SCENE_OFF

The GitHub ESA2 version supports the main seven scenes and does not document the Shaka Gradient scene. The uploaded Casambi version adds Shaka as a Gradient scene.

How Gesture Detection Works

The browser imports MediaPipe HandLandmarker from an online CDN. When the operator clicks Start, the browser requests webcam permission. Once the camera stream is active, the app starts analyzing video frames.

Each frame is passed into MediaPipe. MediaPipe returns hand landmarks, which are normalized coordinate points for the wrist and fingers. The JavaScript does not recognize the hand by image labels alone. It checks landmark geometry.

The system uses helper functions such as:

```
fingerIsUp  
fingerIsDown  
distance  
getHandCenter  
foldedFingerCount
```

These helpers are used to classify the shape of the hand.

An open palm is detected when most fingers are extended upward. A fist is detected when most fingers are folded. A thumbs up checks for folded fingers plus a thumb that is clearly raised and extended. A peace sign checks that the index and middle fingers are up while the ring and pinky fingers are down. A shaka checks for the pinky and thumb out while the other fingers are folded.

Two-hand gestures use comparisons between both hands. A heart gesture checks whether the index fingertips and thumbs are close enough to form a heart-like shape. Jazz hands are detected when both hands are open. Crying can be detected when both hands are closed or when the two hands move near the face in a motion pattern.

The system also includes an inappropriate gesture check. When that gesture is detected, the interface shows a message asking the user to use appropriate gestures instead of sending a lighting command.

Gesture Stabilization

Gesture recognition can flicker because hand tracking changes from frame to frame. To reduce false triggers, HueMotion uses stabilization.

The app stores the last raw gesture and counts how many consecutive frames show the same gesture. A gesture must be stable for enough frames before it becomes an active gesture. Some gestures use fewer frames because they are easier or more important to recognize quickly. Unknown gestures require more frames before the system changes the display back to Unknown.

This prevents the lights from changing because of a single bad frame.

Cooldown

The app also uses a trigger cooldown. After a gesture sends a Casambi command, the same gesture cannot immediately send repeated commands again. This prevents the system from rapidly sending the same scene repeatedly while the user holds the same gesture.

The current cooldown is set to 2500 ms in the uploaded Casambi version.

The Off gesture has extra protection. The user must hold the fist long enough before the Off scene is allowed. The app also blocks accidental Off triggers immediately after some other gestures, such as Green or Blue, to reduce false shutoff events.

Casambi Bridge Operation

When the browser wants to control a scene, it sends a local request like this:

```
/control?key=r
```

The server receives the key and checks whether it is allowed. The allowed keys are:

```
r, g, b, y, w, p, o, x
```

If the key is not allowed, the server returns an error and does not send anything to Casambi.

If the key is allowed, the server makes sure it has a valid Casambi session. It logs in using the configured Casambi API key, email, password, and network ID. After login, it extracts the correct session ID from the response.

Then the server opens a Casambi WebSocket bridge. Once the wire is open, the server can send scene control messages.

The command sent to Casambi has this structure:

```
{
  wire: CASAMBI_WIRE,
  method: "controlScene",
  id: sceneId,
  level: 1
}
```

The id value comes from the matching scene variable in .env.

For example, if the user performs Heart, the browser maps Heart to r, the server maps r to CASAMBI_SCENE_RED, and the server sends that scene ID to Casambi.

Server Routes

The Node.js server uses these routes:

Route	Purpose
/	Serves the HueMotion browser interface.
/logo.jpg	Serves the LUX Dynamics logo file.
/control?key=<key>	Sends a validated lighting scene command.
Other paths	Return a JSON 404 response.

The server also responds to OPTIONS requests for browser compatibility.

Operator Instructions

Start the local server:

```
node server.casambi.js
```

Open:

```
http://localhost:3000
```

Click Start.

Allow camera access.

Stand or sit where both hands are clearly visible.

Perform a supported gesture.

Confirm that the Detected Gesture panel changes.

Confirm that the Lighting Scene panel changes.

Confirm that the RGBW Output panel changes.

Only after the browser preview is behaving correctly, click Enable Casambi.

Perform the gesture again to send the scene to the Casambi network.

Click Disable Casambi before testing uncertain gestures.

Click Stop when finished.

Demonstration Procedure

First, open the HueMotion page with Casambi triggers off. Show that the camera is not yet active.

Second, click Start and allow the camera. Show the tracking overlay and explain that the green hand skeleton is the detected hand landmark structure.

Third, demonstrate each gesture with only the browser preview. Start with Open Palm for White because it is easy to recognize. Then show Thumbs Up for Green, Heart for Red, Peace Sign for Purple, and Fist for Off.

Fourth, click Enable Casambi. Explain that the browser is now allowed to send scene triggers through the local bridge.

Fifth, repeat the gestures and show the physical lights changing.

Sixth, click Disable Casambi or Stop at the end of the demo.

Test Procedure

Test	Expected Result
Run npm install	ws dependency installs without errors.
Run server	Terminal shows local server is running.
Open page	HueMotion interface loads.
Logo loads	LUX Dynamics logo appears.
Click Start	Browser asks for camera permission.
Allow camera	Live video appears.
Show open palm	Detected gesture changes to Open Palm.
Show thumbs up	Detected gesture changes to Thumbs Up.
Show peace sign	Detected gesture changes to Peace Sign.
Show fist	Detected gesture changes to Fist or Off after hold.
Enable Casambi	Button changes to Disable Casambi.
Trigger Red	Casambi receives Red scene.
Trigger Off	Casambi receives Off scene only after valid hold.
Invalid key test	/control?key=z returns an invalid key error.

Troubleshooting

Page does not load

Make sure the server is running. Do not open the JavaScript file directly. The project must be served through the local Node.js server.

Check that the terminal is in the correct project folder.

Run:

```
node server.casambi.js
```

Then open:

`http://localhost:3000`

Logo does not appear

The server expects the logo file to be named:

`logo.jpg`

Rename the uploaded logo file if needed and keep it in the same folder as the server file.

Camera does not start

Check browser permissions. The browser must be allowed to use the camera.

Use Chrome or Edge.

Close other applications that may already be using the camera.

Reload the page and press Start again.

Hand tracking does not work

Make sure the computer has internet access. The browser loads MediaPipe from online sources.

Check browser developer tools for JavaScript errors.

Make sure the camera feed is visible and not black.

Gestures are inaccurate

Improve lighting.

Move hands fully into the camera frame.

Avoid moving too quickly.

Use a plain background.

Hold each gesture for a moment.

Keep hands separated when performing two-hand gestures unless the gesture requires the hands to be close, such as Heart.

Casambi button works, but lights do not change

Check that the `.env` file exists.

Check that the Casambi API key, email, password, and network ID are correct.

Check that the Casambi scene IDs match the scenes in the Casambi network.

Check that the computer has internet access.

Restart the server after changing .env.

Server says missing .env values

The server requires these values:

CASAMBI_API_KEY
CASAMBI_EMAIL
CASAMBI_PASSWORD
CASAMBI_NETWORK_ID

Add the missing values to .env, then restart the server.

Server says Casambi login failed

The API key, email, password, or account permissions may be wrong.

Rotate the credentials if the uploaded .env was shared or committed.

Confirm that the Casambi account has access to the selected network.

Server says Casambi WebSocket is not open

The WebSocket wire may not have opened successfully.

Restart the server.

Confirm internet access.

Confirm the network ID.

Wait a few seconds after the first trigger because the first command may need to create the session and open the wire.

Wrong scene triggers

Check the scene ID values in .env.

For example, if Heart is supposed to trigger Red, confirm that CASAMBI_SCENE_RED is the actual Red scene ID in the Casambi network.

Security and Safety Rules

Do not commit .env.

Do not expose the Casambi API key.

Do not expose the Casambi password.

Rotate credentials if the .env file was uploaded, shared, or committed.

Keep the server local unless there is a controlled reason to expose it.

Do not allow the /control endpoint to accept arbitrary commands. It should only allow known lighting keys.

Keep Casambi triggering off until the camera preview is working.

Use a physical lighting override or access to the Casambi app during demos in case the system needs to be stopped manually.

Development Notes

The uploaded Casambi version contains the HTML, CSS, browser JavaScript, server logic, and Casambi bridge in one file. This makes the demo easy to run, but it also makes the file large.

For long-term maintenance, the project should be split into separate files:

```
server.casambi.js
public/index.html
public/styles.css
public/app.js
public/logo.jpg
.env.example
README.md
```

This would make it easier to edit the interface without touching server code.

The project should also include a safe .env.example file:

```
CASAMBI_API_KEY=
CASAMBI_EMAIL=
CASAMBI_PASSWORD=
CASAMBI_NETWORK_ID=
CASAMBI_WIRE=1

CASAMBI_SCENE_RED=
CASAMBI_SCENE_GREEN=
CASAMBI_SCENE_BLUE=
CASAMBI_SCENE_YELLOW=
CASAMBI_SCENE_WHITE=
CASAMBI_SCENE_PURPLE=
CASAMBI_SCENE_OFF=
CASAMBI_SCENE_GRADIENT=
```

The real .env should stay private.

Legacy ESA2 Version Notes

The GitHub repository documents an earlier ESA2 version. That version uses Node.js to serve the page and uses AutoHotkey to send keyboard shortcuts to ESA2. The supported keys in that version are:

r, g, b, y, w, p, o

The server validates the key, finds AutoHotkey, and runs send_key.ahk. The AutoHotkey script checks that the key is allowed, searches for an ESA2 or Easy Stand-Alone window, and sends the key to ESA2.

That version is useful if the lighting system is controlled through ESA2. For the current Casambi version, the README should be updated because the uploaded project no longer relies on AutoHotkey as the main lighting bridge.

References

Run the system:

```
npm install
node server.casambi.js
```

Open:

<http://localhost:3000>

Main controls:

Button	Function
Start	Starts webcam and hand tracking.
Stop	Stops webcam and resets gesture state.
Enable Casambi	Allows gestures to trigger real Casambi scenes.
Disable Casambi	Stops real scene triggering but keeps preview active.

Gesture keys:

r = Red
g = Green
b = Blue
y = Yellow
w = White
p = Purple
o = Off
x = Gradient